

41052 - Advanced Algorithms
Programming Assignment 1 - Fast Fourier Transform

Tung Nguyen - 24666048

Contents

1	A little history	2
2	Relevant background knowledge	2
2.1	Discrete Fourier Transform	2
2.2	Fast Fourier Transform	3
2.2.1	Iterative Radix-2 FFT	3
2.2.2	Recursive Radix-2 FFT	6
2.2.3	Optimising with Precomputed Twiddle	8
2.2.4	Mixed Radix Fast Fourier Transform	9
3	Code Implementation	11
3.1	Architecture & Repository Navigation	11
3.2	Data Representation & Hardware Cache Alignment	12
3.3	Practical changes	12
3.3.1	Reusing FFT for IFFT	12
3.3.2	Code wrappers	13
3.3.3	Smart finding factor function	13
3.3.4	Catching error	14
4	The Experiments	14
5	The Empirical Study	14
6	What I learned	14
7	AI usage Declaration	14

1 A little history

The Discrete Fourier Transform wasn't invented by Joseph Fourier. It actually predated Fourier's work on the continuous Fourier Transform (formally published in 1822)! In 1759, Alexis Clairaut published an early discrete trigonometric formula while calculating celestial orbits [3]. In 1805, Carl Friedrich Gauss refined this technique to track the paths of asteroids, effectively devising both the discrete Fourier formula and an early Fast Fourier Transform (FFT) [2]. However, his work remained largely overlooked until long after his death because it was recorded in unpublished Latin notes with unconventional notation. Since this formula handles discrete values with methods like the Fourier transform, later on it was named as the Discrete Fourier Transform.

The modern breakthrough occurred during the Cold War. As the United States and the Soviet Union negotiated nuclear test bans, verifying compliance required distinguishing clandestine underground nuclear tests from natural earthquakes [2]. Processing the flood of discrete seismic sensor data was computationally impractical using standard methods, which scaled at $\mathcal{O}(N^2)$.

Physicist and defence adviser Richard Garwin recognised this bottleneck and raised it with statistician John Tukey from Princeton and Bell Labs. Tukey proposed a divide-and-conquer approach to accelerate the computation and worked with IBM mathematician James Cooley, who implemented and programmed the algorithm. Published in 1965, the Cooley–Tukey FFT slashed the computational cost to $\mathcal{O}(N \log_2 N)$ [1] (fundamentally, Gauss invented the same algorithm but his work went unnoticed). But by that time, major countries has already created their nuclear devices and went ahead with their own testing.

Nowadays, the FFT has revolutionised digital signal processing, telecommunications and numerical computing worldwide, thus becoming one of the most important algorithm in our lives.

2 Relevant background knowledge

This project will be implementing and studying seven relevant algorithms.

1. Standard Discrete Fourier Transform
2. Iterative Radix-2 Fast Fourier Transform (FFT)
3. Recursive Radix-2 FFT
4. Iterative Radix-2 with Precomputed Twiddle
5. Mixed Radix FFT

This section will discuss the theoretical background of the above mentioned methods, mainly how to implement the algorithms and their complexity.

2.1 Discrete Fourier Transform

The Discrete Fourier Transform (DFT) is a function that receives an array of complex numbers $x = (x_0, x_1, \dots, x_{N-1})$ and returns a transformed frequency array $X = (X_0, X_1, \dots, X_{N-1})$. Formally, the DFT is defined as:

$$X_k = \sum_{j=0}^{N-1} x_j \cdot W, \quad (1)$$

with $W = \cos(\theta) - i \sin(\theta)$, and $\theta = \frac{2\pi k j}{N}$. The DFT can be defined as an algorithm.

Algorithm 1 Discrete Fourier Transform

Input: Array $x[0 \dots N-1]$ of complex numbers, length N

Output: Transformed array $X[0 \dots N-1]$

```
1: procedure DFT( $x, N$ )
2:   Allocate array  $X[0 \dots N-1]$  initialised to 0
3:   for  $k \leftarrow 0$  to  $N-1$  do
4:     for  $j \leftarrow 0$  to  $N-1$  do
5:        $\theta \leftarrow \frac{2\pi}{N} \cdot k \cdot j$ 
6:        $W \leftarrow \cos \theta - i \sin \theta$ 
7:        $X[k] \leftarrow X[k] + x[j] \cdot W$ 
8:   return  $X$ 
```

The W is known as the twiddle factor

Looking deeper into this algorithm, the first for loop (line 2) is going over every element of the input array of size N (line 6, with $X[k]$). The nested for loop (line 3) is also looping over size N to compute for each value of array X . Thus, the DFT has a time complexity of $\mathcal{O}(N^2)$. For simplicity sake, we can interpret an abstraction of the DFT as follows:

Algorithm 2 Discrete Fourier Transform in a nut shell

Input: Array $x[0 \dots N - 1]$ of complex numbers, length N

Output: Transformed array $X[0 \dots N - 1]$

```

1: procedure DFT( $x, N$ )
2:   Allocate array  $X[0 \dots N - 1]$  initialised to 0
3:   for  $k \leftarrow 0$  to  $N - 1$  do
4:     for  $j \leftarrow 0$  to  $N - 1$  do
5:        $x[j] \rightarrow$  Black box  $\rightarrow X[k]$ 
6:   return  $X$ 

```

Now, we can interpret the DFT as just a for loop, followed by a for loop, both looping over an array. The operation at line 5 does not matter to us (only keep this in mind for simpler explanation why the following FFT works). Notice that for each k , there are N sub-results.

The Inverse Discrete Fourier Transform (IDFT) is used for reversing a transformed array to its original array, essentially undo the DFT. It can also be described as an algorithm.

Algorithm 3 Inverse Discrete Fourier Transform

Input: Transformed array $X = [X[0], X[1], \dots, X[N - 1]]$ of size N

Output: Original array $x = [x[0], x[1], \dots, x[N - 1]]$

```

1: procedure IDFT( $x, N$ )
2:   Allocate array  $x[0 \dots N - 1]$  initialised to 0
3:   for  $j \leftarrow 0$  to  $N - 1$  do
4:     for  $k \leftarrow 0$  to  $N - 1$  do
5:        $\theta \leftarrow \frac{2\pi}{N} \cdot k \cdot j$ 
6:        $W \leftarrow \cos(\theta) + i \sin(\theta)$ 
7:        $x[j] \leftarrow x[j] + X[k] \cdot W$ 
8:    $x[j] \leftarrow \frac{1}{N} \cdot x[j]$ 
9:   return  $x$ 

```

Once again, due to the nested for loops (line 2 and 3), the IDFT has a complexity of $\mathcal{O}(N^2)$. Moreover, it is true that

$$\text{IDFT}(x) = \frac{1}{N} \cdot \text{conj}[\text{DFT}(\text{conj}(x))]. \quad (2)$$

In this property, $\text{conj}()$ is a function that turn a complex number into its complex conjugate (flipping the operator of the imaginary part). This operation and the normalisation over N (line 8) is looping over N input of the array, hence it's complexity is $\mathcal{O}(N)$ and is bounded by the above mentioned $\mathcal{O}(N^2)$. Simply speaking, for all of the mentioned algorithm, the IFFT is just the FFT with different factors involved due to the $\text{conj}()$.

2.2 Fast Fourier Transform

The Fast Fourier Transform is a class of algorithms that can compute the DFT and IDFT faster than $\mathcal{O}(N^2)$. In addition, as mentioned in equation 2, when computing for IDFT, a new complexity of $\mathcal{O}(N)$ is added. Since FFT essentially compute DFT, Inverse Fast Fourier Transform (IFFT) can be used to compute IDFT. And for all of the following algorithms, the $\text{conj}()$ complexity is also bounded.

2.2.1 Iterative Radix-2 FFT

The iterative version runs with the help of the Bit Reversal Permutation (BRP) algorithm. The BRP

essentially loop over an array, for each index it flips the binary (e.g. $001 \mapsto 100, 110 \mapsto 011$), and move the current value to its new index.

Algorithm 4 Bit-Reversal Permutation

Input: Array $A[0 \dots N-1]$, length $N = 2^\ell$

Output: Array A permuted in-place into leaf order

```

1: procedure BIT-REVERSE-PERMUTATION( $A, N$ )
2:    $\ell \leftarrow \log_2 N$ 
3:   for  $i \leftarrow 0$  to  $N-1$  do
4:      $rev \leftarrow 0$ 
5:      $temp \leftarrow i$ 
6:     for  $b \leftarrow 0$  to  $\ell-1$  do
7:        $bit \leftarrow temp \bmod 2$                                 # Extract binary split parity
8:        $rev \leftarrow 2 \cdot rev + bit$                         # Shift accumulated bits left and append
9:        $temp \leftarrow \lfloor temp/2 \rfloor$                         # Advance to next binary digit
10:    if  $i < rev$  then
11:       $swap(A[i], A[rev])$ 

```

Table 1: Bit-reversal permutation mapping for $N = 8$ ($\ell = 3$ bits).

Original Index (i)	Binary ($b_2b_1b_0$)	Reversed ($b_0b_1b_2$)	Permuted Index ($rev(i)$)	Action
0	000	000	0	Identity
1	001	100	4	Swap (1, 4)
2	010	010	2	Identity
3	011	110	6	Swap (3, 6)
4	100	001	1	(Handled at $i = 1$)
5	101	101	5	Identity
6	110	011	3	(Handled at $i = 3$)
7	111	111	7	Identity

Algorithm 5 Iterative Radix-2 FFT

Input: Array $x[0 \dots N-1]$ of complex numbers, length $N = 2^\ell$

Output: Array $x[0 \dots N-1]$ transformed in-place

```

1: procedure FFT( $x, N$ )
2:   BIT-REVERSE-PERMUTATION( $x, N$ )
3:   for  $s \leftarrow 1$  to  $\log_2 N$  do
4:      $m \leftarrow 2^s$ 
5:      $\theta \leftarrow \frac{2\pi}{m}$ 
6:      $W_m \leftarrow \cos(\theta) - i \sin(\theta)$ 
7:     for  $k \leftarrow 0$  to  $N-1$  step  $m$  do
8:        $W \leftarrow 1$ 
9:       for  $j \leftarrow 0$  to  $m/2-1$  do
10:         $t \leftarrow W \cdot x[k+j+m/2]$ 
11:         $u \leftarrow x[k+j]$ 
12:         $x[k+j] \leftarrow u+t$ 
13:         $x[k+j+m/2] \leftarrow u-t$ 
14:         $W \leftarrow W \cdot W_m$ 
15:   return  $x$ 

```

Equivalence

To see why this FFT is equivalent to the DFT, we need to abstract according to algorithm 2. With the BRP, the array is laid out in the form of leaf nodes of the recursion tree sequentially in memory so we can build upward with simple loops. From the example in Table 1, we can have

$$\{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\} \mapsto \{x_0, x_4, x_2, x_6, x_1, x_5, x_3, x_7\}.$$

Notice that $m = 2^s$ with s starting from 1. Visually, at $s = 1, m = 2$, we have

$$(x_0 \oplus x_4), (x_2 \oplus x_6), (x_1 \oplus x_5), (x_3 \oplus x_7)$$

We know that the first pair is using indices 0 and 1, the second is using 2 and 3, and so on. So each pair is using an even and an odd index. In each pair, the first element acts as the *even* sub-result and the second acts as the *odd* sub-result. Applying a single black box operation to each pair combines these four adjacent pairs into four independent 2-point DFT solutions, fully updated in-place.

Next, at $s = 2$ ($m = 4$), the problem size doubles. The algorithm *merges* adjacent pairs of these 2-point solutions into 4-point solutions:

$$([x_0, x_4] \oplus [x_2, x_6]), ([x_1, x_5] \oplus [x_3, x_7])$$

Here, the first pair x_0, x_4 represents the even-indexed frequency components, while x_2, x_6 represents the odd ones. The loop runs two butterflies ($j = 0, 1$) across an offset of $m/2 = 2$ slots in memory, updating each block into a valid 4-point DFT.

Finally, at $s = 3$ ($m = 8$), the algorithm performs the last merge step:

$$([x_0, x_4, x_2, x_6] \oplus [x_1, x_5, x_3, x_7])$$

The left block contains all even values of the original array, and the right block contains all odd values. Running four butterflies ($j = 0, 1, 2, 3$) across an offset of $m/2 = 4$ combines them into the final, complete 8-point DFT array.

Since each stage simply merges smaller, already-transformed sub-arrays into larger ones (same as bottom-up Merge Sort). The loops mathematically evaluate the exact same divide-and-conquer tree as the recursive DFT definition.

Moving on, on line 12 and 13, we can see that they are $u + t$ and $u - t$, respectively. To understand this, we need to work on the original DFT in equation 1.

$$\begin{aligned} X_k &= \sum_{j=0}^{N-1} x_j \cdot W \\ &= \sum_{m=0}^{N/2-1} x_{2m} \cdot W_N^{2mk} + \sum_{m=0}^{N/2-1} x_{2m+1} \cdot W_N^{(2m+1)k} \quad (\text{Splitting into even and odd halves}) \\ &= \sum_{m=0}^{N/2-1} x_{2m} \cdot (W_N^2)^{mk} + \sum_{m=0}^{N/2-1} x_{2m+1} \cdot W_N^k \cdot W_{N/2}^{mk} \\ &= \sum_{m=0}^{N/2-1} x_{2m} \cdot W_{N/2}^{mk} + W_N^k \cdot \sum_{m=0}^{N/2-1} x_{2m+1} \cdot W_{N/2}^{mk} \\ &= E_k + W_N^k \cdot O_k. \end{aligned} \tag{3}$$

We can see that $u \equiv E_k$ since $k + j$ is the first of the pair, which is even. And, $t \equiv W_N^k \cdot O_k$ since $k + j + m/2$ is the second of the pair, which is odd. Thus, based on line 13, we have

$$X_{k+m/2} = E_{k+m/2} - W_N^k \cdot O_k.$$

We then have

$$E_{k+m/2} = \sum_{m=0}^{N/2-1} x_{2m} \cdot W_{N/2}^{m(k+m/2)}.$$

Specifically,

$$W_{N/2}^{m(k+m/2)} = W_{N/2}^{mk} \cdot (W_{N/2}^{m/2})^m = W_{N/2}^{mk} \cdot 1^m = W_{N/2}^{mk}.$$

Note that after the iteration, which is when this FFT is completed, $m = 2^{\log_2 N} = N$. Thus,

$$W_{N/2}^{m/2} = W_{N/2}^{N/2} = \cos\left(-2\pi \cdot \frac{N}{2}\right) - j \sin\left(-2\pi \cdot \frac{N}{2}\right) = 1.$$

Thus, $E_k = E_{k+m/2}$. Same holds true for $O_k = O_{k+m/2}$. As for W_m^k , we have

$$W_m^{k+m/2} = W_m^k \cdot W_m^{m/2} = W_m^k \cdot \left[\cos\left(2\pi \cdot \frac{m}{2}\right) - j \sin\left(2\pi \cdot \frac{m}{2}\right) \right] = W_m^k \cdot -1 = -W_m^k.$$

Therefore, line 13 is valid.

Complexity

Looking deeper into the Iterative version of the Radix-2 FFT, the first part of the algorithm is spent running the BRP algorithm, which first cost $\mathcal{O}(N)$ looping over the entire array. And a nested for loop inside that cost $\mathcal{O}(\log_2 N)$. In total, the BRP algorithm cost $\mathcal{O}(N \log_2 N)$. Next, the algorithm loops (line 4) over $\log_2 N$ times, followed by a nested loop (line 7) of $\frac{N}{m}$ times (indicated by the **step** m), followed by another nested loop (line 9) of $\frac{m}{2}$ times. In total, this FFT algorithm cost $\mathcal{O}(N \log_2 N + \log_2 N \cdot \frac{N}{m} \cdot \frac{m}{2}) = \mathcal{O}(N \log_2 N)$. Also, since this is in-place, the space complexity only needs to account for the original input, which is $\mathcal{O}(N)$.

Algorithm 6 Iterative Radix-2 IFFT

Input: Transformed array $X[0 \dots N-1]$ of complex numbers, length $N = 2^\ell$

Output: Original array $x[0 \dots N-1]$

```

1: procedure FFT( $x, N$ )
2:    $m \leftarrow \log_2 N$ 
3:   BIT-REVERSE-PERMUTATION( $x, N$ )
4:   for  $s \leftarrow 1$  to  $\log_2 N$  do
5:      $m \leftarrow 2^s$ 
6:      $\theta \leftarrow \frac{2\pi}{m}$ 
7:      $W_m \leftarrow \cos(\theta) + i \sin(\theta)$ 
8:     for  $k \leftarrow 0$  to  $N-1$  step  $m$  do
9:        $W \leftarrow 1$ 
10:      for  $j \leftarrow 0$  to  $m/2-1$  do
11:         $t \leftarrow W \cdot x[k+j+m/2]$ 
12:         $u \leftarrow x[k+j]$ 
13:         $x[k+j] \leftarrow u+t$ 
14:         $x[k+j+m/2] \leftarrow u-t$ 
15:         $W \leftarrow W \cdot W_m$ 
16:      for  $i \leftarrow 0$  to  $N-1$  do
17:         $x[i] \leftarrow \frac{x[i]}{N}$ 
18:      return  $x$ 
```

2.2.2 Recursive Radix-2 FFT

This is simply a recursive version of the above FFT, so the mathematical derivative is the same.

Algorithm 7 Recursive Radix-2 FFT

Input: Array $x[0 \dots N-1]$ of complex numbers, length $N = 2^\ell$

Output: Transformed array $X[0 \dots N-1]$

```
1: procedure FFT( $x, N$ )
2:   if  $N = 1$  then
3:     return  $x$ 
4:   Allocate array  $x_{\text{even}}$  of length  $N/2$ 
5:   Allocate array  $x_{\text{odd}}$  of length  $N/2$ 
6:   for  $k \leftarrow 0$  to  $N/2 - 1$  do
7:      $x_{\text{even}}[k] \leftarrow x[2k]$ 
8:      $x_{\text{odd}}[k] \leftarrow x[2k + 1]$ 
9:    $E \leftarrow \text{FFT}(x_{\text{even}}, N/2)$ 
10:   $O \leftarrow \text{FFT}(x_{\text{odd}}, N/2)$ 
11:  Allocate array  $X$  of length  $N$ 
12:  for  $k \leftarrow 0$  to  $N/2 - 1$  do
13:     $\theta \leftarrow \frac{2\pi}{N} \cdot k$ 
14:     $W \leftarrow \cos(\theta) - i \sin(\theta)$ 
15:     $t \leftarrow W \cdot O[k]$ 
16:     $X[k] \leftarrow E[k] + t$ 
17:     $X[k + N/2] \leftarrow E[k] - t$ 
18:  return  $X$ 
```

Equivalence

The recursive Radix-2 FFT computes the exact same result as the DFT by framing it as a top-down divide-and-conquer problem. Instead of evaluating all frequency components at once, it de-interleaves the input signal into two smaller sequences containing only the even-indexed and odd-indexed samples. Then, it recursively compute the sub-DFTs for each half until reaching the trivial base case of single-element arrays.

Since shifting a sequence in time corresponds to a phase rotation in frequency, the odd sub-transform is multiplied by a twiddle factor to account for its one-sample time offset. Finally, it exploit the circular symmetry of frequencies the $u - t$ proof above). The algorithm reconstructs the full-length DFT by simultaneously computing the first half of the array through addition and the second half through subtraction, precisely reproducing the mathematical sum of the full N -point DFT.

Complexity

The Recursive version of the Radix-2 FFT first copy into two arrays x_{even} and x_{odd} (line 6 to 8), with each costing $\mathcal{O}\left(\frac{N}{2}\right)$. Then, it recursively called with inputs of size $\frac{N}{2}$ two times (line 9 and 10), each time costing $T\left(\frac{N}{2}\right)$ (since it's recursion, we cannot use Big $\mathcal{O}$ right away). Finally, the algorithm loop over $\frac{N}{2}$ times (line 12). Thus, the entire algorithm cost

$$T(N) = 2\mathcal{O}\left(\frac{N}{2}\right) + 2T\left(\frac{N}{2}\right) + \mathcal{O}\left(\frac{N}{2}\right) = \mathcal{O}(N) + 2T\left(\frac{N}{2}\right).$$

According to the Master Theorem, we have

$$T(N) = aT\left(\frac{N}{b}\right) + f(N).$$

It is clear that $f(n) = \mathcal{O}(N)$. Moreover, $a = 2, b = 2 \Rightarrow N^{\log_2 2} = N$. This matches with the case 2 of the Master Theorem. Therefore,

$$T(N) = \mathcal{O}(N \log_2 N).$$

In terms of space complexity, due to the allocation of E and O , both are $N/2$ in length, and X of size N , the space complexity is $\mathcal{O}(N \log_2 N)$.

Algorithm 8 Recursive Radix-2 IFFT

Input: Array $x[0 \dots N-1]$ of complex numbers, length $N = 2^\ell$

Output: Transformed array $X[0 \dots N-1]$

```
1: procedure FFT( $x, N$ )
2:   if  $N = 1$  then
3:     return  $x$ 
4:   Allocate array  $x_{\text{even}}$  of length  $N/2$ 
5:   Allocate array  $x_{\text{odd}}$  of length  $N/2$ 
6:   for  $k \leftarrow 0$  to  $N/2 - 1$  do
7:      $x_{\text{even}}[k] \leftarrow x[2k]$ 
8:      $x_{\text{odd}}[k] \leftarrow x[2k + 1]$ 
9:    $E \leftarrow \text{FFT}(x_{\text{even}}, N/2)$                                 # Sub-transform of even elements
10:   $O \leftarrow \text{FFT}(x_{\text{odd}}, N/2)$                                 # Sub-transform of odd elements
11:  Allocate array  $X$  of length  $N$ 
12:  for  $k \leftarrow 0$  to  $N/2 - 1$  do
13:     $\theta \leftarrow \frac{2\pi}{N} \cdot k$ 
14:     $W \leftarrow \cos(\theta) + i \sin(\theta)$ 
15:     $t \leftarrow W \cdot O[k]$ 
16:     $X[k] \leftarrow E[k] + t$ 
17:     $X[k + N/2] \leftarrow E[k] - t$ 
18:  return  $X$ 
```

2.2.3 Optimising with Precomputed Twiddle

On paper, a small optimisation can be done by pre-computing the twiddles factor (W) first. Since a major part of the FFT is to compute the twiddles, naturally having not to deal with them in the actual algorithm is faster.

Algorithm 9 Iterative Radix-2 FFT with Precomputed Twiddle Table

Input: Array $x[0 \dots N-1]$ of complex numbers, length $N = 2^\ell$, precomputed table $W[0 \dots N/2 - 1]$ where $W[k] = e^{-i\frac{2\pi}{N}k}$

Output: Transformed array $X[0 \dots N-1]$

```
1: procedure PRECOMPUTE-TWIDDLES( $N$ )
2:   Allocate table  $W$  of size  $N/2$ 
3:   for  $k \leftarrow 0$  to  $N/2 - 1$  do
4:      $\theta \leftarrow \frac{2\pi}{N} \cdot k$ 
5:      $W[k] \leftarrow \cos(\theta) - i \sin(\theta)$ 
6:   return  $W$ 

7: procedure FFT( $x, N, W$ )
8:   BIT-REVERSE-PERMUTATION( $X, N$ )
9:   for  $s \leftarrow 1$  to  $\log_2 N$  do                                # Stage loop
10:     $m \leftarrow 2^s$                                               # Sub-transform size
11:     $\text{step} \leftarrow N/m$                                           # Stride to sample twiddles from global table
12:    for  $k \leftarrow 0$  to  $N - 1$  step  $m$  do                        # Block loop
13:      for  $j \leftarrow 0$  to  $m/2 - 1$  do                            # Butterfly loop
14:         $W \leftarrow W[j \cdot \text{step}]$                             #  $\mathcal{O}(1)$  table lookup
15:         $t \leftarrow W \cdot X[k + j + m/2]$ 
16:         $u \leftarrow X[k + j]$ 
17:         $X[k + j] \leftarrow u + t$ 
18:         $X[k + j + m/2] \leftarrow u - t$ 
19:  return  $X$ 
```

This algorithm is essentially the Iterative Radix-2 FFT, with just the twiddles being computed beforehand. The complexity of this algorithm is the same as the Iterative Radix-2 FFT mentioned above, $\mathcal{O}(N \log_2 N)$. The addition complexity of $\mathcal{O}(N)$ introduced from the pre-computation (line 3) is bounded. The auxiliary space here is $\mathcal{O}(N)$ due to the precomputed twiddles.

Algorithm 10 Iterative Radix-2 IFFT with Precomputed Twiddle Table

Input: Array $x[0 \dots N-1]$ of complex numbers, length $N = 2^m$, precomputed table $W[0 \dots N/2-1]$ where $W[k] = e^{-i\frac{2\pi}{N}k}$

Output: Transformed array $X[0 \dots N-1]$

```
1: procedure PRECOMPUTE-TWIDDLES( $N$ )
2:   Allocate table  $W$  of size  $N/2$ 
3:   for  $k \leftarrow 0$  to  $N/2 - 1$  do
4:      $W[k] \leftarrow \cos\left(\frac{2\pi k}{N}\right) + i \sin\left(\frac{2\pi k}{N}\right)$ 
5:   return  $W$ 

6: procedure FFT( $x, N, W$ )
7:    $m \leftarrow \log_2 N$ 
8:   BIT-REVERSE-PERMUTATION( $X, N, m$ )
9:   for  $s \leftarrow 1$  to  $\log_2 N$  do                                     # Stage loop
10:     $m \leftarrow 2^s$                                                 # Sub-transform size
11:     $step \leftarrow N/m$                                              # Stride to sample twiddles from global table
12:    for  $k \leftarrow 0$  to  $N-1$  step  $m$  do                             # Block loop
13:      for  $j \leftarrow 0$  to  $m/2 - 1$  do                               # Butterfly loop
14:         $W \leftarrow W[j \cdot step]$                                 #  $\mathcal{O}(1)$  table lookup
15:         $t \leftarrow W \cdot X[k + j + m/2]$ 
16:         $u \leftarrow X[k + j]$ 
17:         $X[k + j] \leftarrow u + t$ 
18:         $X[k + j + m/2] \leftarrow u - t$ 
19:   return  $X$ 
```

2.2.4 Mixed Radix Fast Fourier Transform

So far, the Radix-2 FFT algorithm only work when the length $N = 2^m$, with $m \in \mathbb{N}$. For array with a length that wouldn't fit that constraint, a General Cooley–Tukey FFT, otherwise known as the Mixed-Radix FFT, can be applied.

Algorithm 11 Mixed-Radix FFT

Input: Sequence $x[0 \dots N-1]$, factors N_1, N_2 such that $N = N_1 \times N_2$ **Output:** Transformed frequency array $X[0 \dots N-1]$

```
1: procedure FFT( $x, N_1, N_2$ )
2:   Allocate 2D matrices  $A[0 \dots N_1-1][0 \dots N_2-1]$  and  $B[0 \dots N_1-1][0 \dots N_2-1]$ 
3:   Allocate 1D array  $X[0 \dots N-1]$ 
4:    $N \leftarrow N_1 \cdot N_2$ 
                                     # Step 1: Map 1D input into  $N_1 \times N_2$  matrix
5:   for  $n_1 \leftarrow 0$  to  $N_1 - 1$  do
6:     for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
7:        $A[n_1][n_2] \leftarrow x[n_1 \cdot N_2 + n_2]$ 
                                     # Step 2:  $N_2$  independent column DFTs of size  $N_1$ 
8:   for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
9:      $col \leftarrow [A[0][n_2], A[1][n_2], \dots, A[N_1-1][n_2]]$ 
10:     $col\_dft \leftarrow \text{DFT}(col, N_1)$ 
11:    for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
12:       $A[k_1][n_2] \leftarrow col\_dft[k_1]$ 
                                     # Step 3: Pointwise twiddle factor multiplication evaluated on the fly
13:   for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
14:     for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
15:        $W \leftarrow \exp(-i \frac{2\pi}{N} \cdot k_1 \cdot n_2)$                                      # Dynamic calculation
16:        $A[k_1][n_2] \leftarrow A[k_1][n_2] \cdot W$ 
                                     # Step 4:  $N_1$  independent row DFTs of size  $N_2$ 
17:   for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
18:      $row \leftarrow [A[k_1][0], A[k_1][1], \dots, A[k_1][N_2-1]]$ 
19:      $row\_dft \leftarrow \text{DFT}(row, N_2)$ 
20:     for  $k_2 \leftarrow 0$  to  $N_2 - 1$  do
21:        $B[k_1][k_2] \leftarrow row\_dft[k_2]$ 
                                     # Step 5: Read out transposed matrix into 1D output
22:   for  $k_2 \leftarrow 0$  to  $N_2 - 1$  do
23:     for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
24:        $X[k_2 \cdot N_1 + k_1] \leftarrow B[k_1][k_2]$ 
25:   return  $X$ 
```

Equivalence

The Mixed-Radix FFT computes the exact N -point DFT by reshaping a composite length $N = N_1 \times N_2$ into a two-dimensional grid and solving it in smaller stages. Instead of transforming all N points in a single pass, the 1D input is written row-by-row into an $N_1 \times N_2$ matrix. The algorithm first computes the DFT along each column (size N_1), decomposing the original signal across that dimension. Next, each intermediate matrix element is scaled by a twiddle factor, which is accounted for the time offset between different columns within the 1D sequence. A second round of DFTs is then performed along each row (size N_2), resolving the remaining dimension. Finally, flattening the matrix column-by-column (transposed order) perfectly reconstructs the original 1D frequency spectrum, achieving an output identical to evaluating the full N -point DFT definition directly.

Complexity

The initial matrix population (Lines 5–9) and the final transposed readout (Lines 29–33) both sweep over all entries in the $N_1 \times N_2$ grid, contributing $\mathcal{O}(N_1 N_2) = \mathcal{O}(N)$ cost.

The algorithm computes N_2 independent sub-DFTs of size N_1 (Lines 11–17). Using direct DFT evaluation, this stage requires $N_2 \cdot \mathcal{O}(N_1^2) = \mathcal{O}(N \cdot N_1)$ operations (or $\mathcal{O}(N \log N_1)$ if solved recursively via FFT).

The double loop (Lines 19–24) multiplies each entry by a twiddle factor, executing exactly $N_1 N_2 = N$ complex multiplications, which is $\mathcal{O}(N)$.

The algorithm executes N_1 independent sub-DFTs of size N_2 (Lines 26–31). Under direct DFT evaluation, this consumes $N_1 \cdot \mathcal{O}(N_2^2) = \mathcal{O}(N \cdot N_2)$ operations (or $\mathcal{O}(N \log N_2)$ via FFT).

Summing these contributions yields a total time complexity of $\mathcal{O}(N(N_1 + N_2))$ when using standard

DFT sub-routines. When N_1 and N_2 are composite and solved recursively using FFT kernels, the total runtime reaches the optimal $\mathcal{O}(N \log N)$. One thing to note is if N is prime, then $N = 1 \cdot N$, which mean this cannot be arranged into a matrix and hence the complexity would just be $\mathcal{O}(N^2)$. In terms of auxiliary space, storing the intermediate matrices A , B , and the output array X (Lines 2–3) requires $\mathcal{O}(N_1 N_2) = \mathcal{O}(N)$ additional memory, and the total space complexity is $\mathcal{O}(N)$.

Algorithm 12 Mixed-Radix IFFT

Input: Transformed frequency array $X[0 \dots N - 1]$, factors N_1, N_2 such that $N = N_1 \times N_2$
Output: Reconstructed time sequence $x[0 \dots N - 1]$

```

1: procedure IFFT( $X, N_1, N_2$ )
2:   Allocate 2D matrices  $A[0 \dots N_1 - 1][0 \dots N_2 - 1]$  and  $B[0 \dots N_1 - 1][0 \dots N_2 - 1]$ 
3:   Allocate 1D array  $x[0 \dots N - 1]$ 
4:    $N \leftarrow N_1 \cdot N_2$ 
                                     # Step 1: Map 1D frequency input into transposed  $N_1 \times N_2$  matrix
5:   for  $k_2 \leftarrow 0$  to  $N_2 - 1$  do
6:     for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
7:        $B[k_1][k_2] \leftarrow X[k_2 \cdot N_1 + k_1]$ 
                                     # Step 2:  $N_1$  independent row IDFTs of size  $N_2$ 
8:   for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
9:      $row \leftarrow [B[k_1][0], B[k_1][1], \dots, B[k_1][N_2 - 1]]$ 
10:     $row\_idft \leftarrow \text{IDFT}(row, N_2)$ 
11:    for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
12:       $A[k_1][n_2] \leftarrow row\_idft[n_2]$ 
                                     # Step 3: Pointwise inverse twiddle factor multiplication
13:  for  $k_1 \leftarrow 0$  to  $N_1 - 1$  do
14:    for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
15:       $W \leftarrow \exp(+i \frac{2\pi}{N} \cdot k_1 \cdot n_2)$                                      # Positive sign for inverse
16:       $A[k_1][n_2] \leftarrow A[k_1][n_2] \cdot W$ 
                                     # Step 4:  $N_2$  independent column IDFTs of size  $N_1$ 
17:  for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
18:     $col \leftarrow [A[0][n_2], A[1][n_2], \dots, A[N_1 - 1][n_2]]$ 
19:     $col\_idft \leftarrow \text{IDFT}(col, N_1)$ 
20:    for  $n_1 \leftarrow 0$  to  $N_1 - 1$  do
21:       $A[n_1][n_2] \leftarrow col\_idft[n_1]$ 
                                     # Step 5: Read out matrix into 1D time sequence
22:  for  $n_1 \leftarrow 0$  to  $N_1 - 1$  do
23:    for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
24:       $x[n_1 \cdot N_2 + n_2] \leftarrow A[n_1][n_2]$ 
25:  return  $x$ 

```

3 Code Implementation

This section will discuss about how the pseudocodes are coded. To keep it concise, if the code is a direct one-to-one match with the pseudocodes without being too unreadable, then it will be skipped.

Programming Language Choice

The main language to implement the algorithm is C++. The reason for this is it is a compiled language, which mean the runtime would be more consistent than interpret language. It also does not have a garbage collector, so no runtime boxing of floats. Finally, it just very fast and low level, which can capture the cache performance better. Python is chosen as the benchmarking language due to its ability to draw graphs and easy input/output management.

3.1 Architecture & Repository Navigation

Within implementation/:

- `include/`: This is where the headers and clean interfaces live.

- **src/**: The actual C++ implementation of the above mentioned algorithms.
- **tests/**: Automated unit tests and correctness validation.
- **bench/**: The experiments and results.

To represent complex numbers, the type `Complex` declared.

3.2 Data Representation & Hardware Cache Alignment

The type declaration can be found in `include/types.hpp`.

```
1 using Complex = std::complex<double>;
2 using ComplexVector = std::vector<Complex>;
```

The primary array container is defined as:

```
using ComplexVector = std::vector<std::complex<double>>;
```

Under the ISO C++ standard (§26.4), the class template `std::complex<T>` is guaranteed to be binary layout-compatible with a two-element array `T[2]`. Specifically, the imaginary part immediately follows the real part in memory with zero padding. Combined with `std::vector`, which allocates elements in a single contiguous heap buffer, the entire signal of length N is laid out linearly:

$$\text{Memory Buffer} = \underbrace{[\text{Re}_0, \text{Im}_0]}_{\text{Element 0 (16 B)}} \underbrace{[\text{Re}_1, \text{Im}_1]}_{\text{Element 1 (16 B)}} \underbrace{[\text{Re}_2, \text{Im}_2]}_{\text{Element 2 (16 B)}} \dots \underbrace{[\text{Re}_{N-1}, \text{Im}_{N-1}]}_{\text{Element } N-1 \text{ (16 B)}} \quad (4)$$

Moreover, on x86-64 processors, the L1, L2, and L3 data caches operate on chunks of 64 bytes (a single cache line). Since each `std::complex<double>` occupies:

$$2 \times 8 \text{ bytes (IEEE 754 double precision)} = 16 \text{ bytes}, \quad (5)$$

a single 64-byte cache line holds exactly 4 contiguous complex samples:

Sample k (16 B)	Sample $k + 1$ (16 B)	Sample $k + 2$ (16 B)	Sample $k + 3$ (16 B)
← One 64-Byte CPU Cache Line →			

In the early stages of the iterative FFT (e.g., $m = 2$ and $m = 4$), butterfly operations access adjacent indices (k and $k + 1$). When the CPU reads sample k from memory, the hardware cache line fills and prefetchers automatically bring samples $k + 1$, $k + 2$, and $k + 3$ into L1 cache with zero additional latency. Moreover, in languages such as Java or Python, an array of complex numbers is typically an array of object pointers, requiring an extra pointer dereference per access and scattering memory allocations across the heap. The flat C++ representation ensures purely contiguous memory access without heap boxing.

3.3 Practical changes

Given the above mentioned pseudocodes as the textbook definitions, several changes were made in the code to make it more practical.

3.3.1 Reusing FFT for IFFT

In `src/mixed-radix_fft.cpp`

```
1 ComplexVector conjugated(n);
2 ComplexVector conjugated(n);
3 for (std::size_t i = 0; i < n; ++i) {
4     conjugated[i] = std::conj(input[i]);
5 }
6 ComplexVector res = mixed_radix_fft(conjugated);
```

and `recursive_radix-2_fft.cpp`,

```

1 ComplexVector conjugated(n);
2 for (std::size_t i = 0; i < n; ++i) {
3     conjugated[i] = std::conj(input[i]);
4 }
5 ComplexVector res = recursive_radix2_fft(conjugated);

```

Instead of rewriting a entire new function to compute IFFT, these two files just compute the conjugate of the array, pass them through the FFT (which function the same), and then normalise each element. This reduce code redundancy. However, this approach requires two additional $\mathcal{O}(N)$ passes across the array (one to compute the conjugate, one to normalise the array). For large datasets that exceed the CPU cache capacity, traversing the entire sequence in memory two extra times increase memory traffic. So a single pass via a dedicate IFFT function would reduce the load on the cache capacity. The reason for this is because for the two mentioned algorithm, they both use recursion, which already allocates arrays per call, the conjugate can eliminate code duplication. Particularly, for the Mixed Radix FFT, rewrite the entire long function was quite redundant.

3.3.2 Code wrappers

In `src/mixed_radix_fft.cpp` and `precomputed_twiddle_fft.cpp`, there are function with the same name but different input. For example,

```

1 ComplexVector precomputed_twiddle_fft(const ComplexVector& input, const ComplexVector& W) {
2     \\ Long code
3 }
4
5 ComplexVector precomputed_twiddle_fft(const ComplexVector& input) {
6     \\ Shorter code
7 }

```

Heuristically, the one function with less input and shorter code is the wrapper. They help with being convenient when called, and the main function (the longer code with more input) can be read easier. Moreover, it is simply a single pass with no copy, so it does not worsen memory passes in this case.

3.3.3 Smart finding factor function

In `src/mixed_radix_fft.cpp`,

```

1 std::size_t find_smallest_factor(std::size_t n) {
2     if (n <= 3)
3         return n;
4     if (n % 2 == 0)
5         return 2;
6     if (n % 3 == 0)
7         return 3;
8
9     // According to the division algorithm theorem,
10    // for every 6 numbers, 4 of them are composite
11    for (std::size_t i = 5; i * i <= n; i += 6) {
12        if (n % i == 0)
13            return i;
14        if (n % (i + 2) == 0)
15            return i + 2;
16    }
17    return n; // n is prime
18 }

```

To compute this algorithm, we must decompose N into two factors $N = N_1 \times N_2$, and the most efficient approach is to choose N_1 as the smallest prime factor of N . This prevents redundant trial division by composite multiples (for example, if N is not divisible by 2, testing 4 or 6 is unnecessary).

According to the division algorithm, all integers can be expressed in the form $6k + r$ where $r \in \{0, 1, 2, 3, 4, 5\}$. For any integer k , numbers with remainders 0, 2, 3, and 4 are divisible by 2 or 3 and

are therefore guaranteed to be composite for values greater than 3. Consequently, all prime numbers greater than 3 must take the form $6k \pm 1$. Exploiting this property, the loop skips the four guaranteed composite residue classes in each block of six and only tests the two potential prime candidates (i and $i + 2$), stepping by 6 each iteration.

3.3.4 Catching error

Throughout the codes in `src/`, there are various safe guard to guarantee that the algorithm can proceed according to its constraint smoothly. For example in `iterative_radix-2_fft.cpp`,

```
1  if (!is_power_of_two(n)) {  
2      throw std::invalid_argument("Input array size must be a power of 2 for Radix-2 FFT");  
3  }
```

4 The Experiments

5 The Empirical Study

6 What I learned

7 AI usage Declaration

References

- [1] James W. Cooley and John W. Tukey. 1965. An Algorithm for the Machine Calculation of Complex Fourier Series. *Math. Comp.* 19, 90 (1965), 297–301. <http://www.jstor.org/stable/2003354>
- [2] Michael T. Heideman, Don H. Johnson, and C. Sidney Burrus. 1985. Gauss and the History of the Fast Fourier Transform. *Archive for History of Exact Sciences* 34, 3 (1985), 265–277. <http://www.jstor.org/stable/41133773>
- [3] Audrey Terras. 1999. *Fourier Analysis on Finite Groups and Applications*. Cambridge University Press.